Fast Approximate Vector Joins via Offline-Online Co-Design

Kyoungmin Kim
EPFL
kyoung-min.kim@epfl.ch

Lennart Roth
EPFL
lennart.roth@epfl.ch

Liang Liang
EPFL
liang.liang@epfl.ch

Anastasia Ailamaki
EPFL
anastasia.ailamaki@epfl.ch

arXiv:2603.16360v2 [cs.DB] 23 Jul 2026

ABSTRACT

Approximate vector join is a fundamental primitive for AI-driven data management. Prior methods reduce online search cost through work sharing across queries, but remain limited by hard reuse of only final join results, separate query/data-side indexes over vectors that already live in a shared embedding space, and recall loss on difficult queries that lie far from data distributions. We present an offline–online co-design for approximate vector join that addresses these limitations through three complementary ideas. First, we introduce an encoder-centric merged index that jointly organizes vectors from the same embedding space and offloads the main bottleneck of vector join to offline index construction. Second, we propose soft work sharing that generalizes reuse from final join results to useful search effort. Third, we develop an adaptive hybrid search that selectively improves recall for difficult queries while avoiding unnecessary overhead on easy ones. Together, these techniques provide a modular yet principled design that improves both efficiency and robustness across diverse datasets. By treating the underlying graph index as a black box, it can generalize across arbitrary graph indexes.

1 INTRODUCTION

Similarity joins over vector embeddings are a fundamental operation in modern AI-driven data management and analytics [34, 43]. Many real-world applications require identifying all pairs of semantically similar objects across two collections, rather than retrieving only the top-k nearest neighbors for each query [12]. An example is cross-source product matching, where two catalogs are encoded by the same embedding model and threshold-based vector join retrieves all candidate pairs (detailed in Appendix A) [30, 40]. Near-duplicate detection in image, video, or document collections relies on self-joins to find all items whose embeddings fall within a small distance threshold [43]. In data integration and record linkage, vector joins are used to match entities across heterogeneous sources, such as joining product descriptions, customer profiles, or scientific records based on semantic similarity [14, 44]. In recommendation, fraud detection, and security analytics, vector joins enable discovering clusters of closely related behaviors or anomalously similar activity patterns [43]. They also underlie hybrid vector-relational queries, enabling filters, joins, and aggregations over both symbolic and semantic attributes [1, 10, 13, 36, 48].

Given two sets of vectors X and Y (assume $|X| \leq |Y|$) and a distance threshold θ, the goal of threshold-based vector join is to find all similar vector pairs (x, y) such that $x \in X, y \in Y, dist(x, y) < \theta$ for a distance function $dist(\cdot, \cdot)$. We focus on the *in-memory* vector-join setting, where the primary bottleneck is distance computation over high-dimensional vectors rather than disk I/O [12, 33]. Accordingly, our goal is to reduce the amount of online distance computations for candidate pairs.

A straightforward way to evaluate threshold-based vector joins is to probe an approximate nearest neighbor (ANN) index over the larger set Y (simply data) independently for each vector in X (simply query), i.e., an index nested-loop style execution [1, 10, 35]. As in top-k search, graph-based indexes such as HNSW [32] and NSG [17] have been used due to their impressive performance [44, 45]. Then, for each query, a threshold-based search first reaches an *in-range region*, a data region containing points within distance θ of the query (via greedy search), and then expands within that region to collect all matches (post-greedy expansion) [45].

We observe that the greedy search can take up to 99% of the total time especially under small thresholds (Figure 10), traversing a large portion of the index before hitting an in-range point. Moreover, each search starts from a designated index entry, resulting in *redundant* traversals that could have been reused for similar queries. Recent work [45] aims to mitigate this through *work sharing*: similar queries are processed in a certain order (obtained from a query-side index) so that a later query can reuse information from an earlier one. However, this has been only done in *hard* form, reusing only the final in-range points found as the seeds to later query traversals. This makes reuse restrictive, since useful effort spent reaching nearby but out-of-range (simply out-range) points is discarded. Moreover, since query- and data-side indexes are maintained separately, the online phase must rediscover the proximity relationships between the two vector sets from scratch for every join operation, incurring repetitive overhead. Finally, even after reaching an in-range region, traversing only within that region [45] may miss other disconnected in-range regions and lose recall. This is pronounced for out-of-distribution (OOD) queries where the query and data distributions barely overlap, and closest data points to a query are scattered over the data space [11].

We can then summarize these into three orthogonal issues: (1) *redundancy across similar queries* that has been addressed only in a limited way [45], (2) *redundancy across joins* that repeatedly rediscovers query-data relationships, and (3) *reachability failure on difficult queries* that, even after reaching one in-range region, the search may fail to reach other disconnected in-range regions.

This paper addresses these issues through a unified offline–online co-design, organized around a single principle: shift as much join effort as possible to offline index construction, and handle only what cannot be anticipated offline at query time, as efficiently and

robustly as possible, while remaining simple and index-agnostic. This principle naturally gives rise to three components. To mitigate (2), we propose a merged index (offline) that organizes vectors from a shared embedding space in one unified graph, eliminating the need to rediscover cross-set geometry at query time and offloading initial in-range region discovery to index construction in a *threshold-agnostic* way. To mitigate (1), we introduce soft work sharing (online), which generalizes reuse from only final join results to useful search effort produced during traversal (closest data point found so far, which may be out-range). Importantly, soft work sharing remains beneficial even when merged indexing is unavailable as it directly reduces redundant online traversal across similar queries regardless of index organization. To mitigate (3), we propose an OOD-aware adaptive hybrid search (online) that traverses out-range points as bridges to disconnected in-range regions, but only for queries predicted OOD using a lightweight query-data overlap signal provided by the merged index. Each component addresses what the previous one leaves unsolved: the merged index eliminates redundant rediscovery of cross-set geometry, soft work sharing maximizes reuse of remaining online traversal, and hybrid search recovers the recall that neither preparation nor reuse can guarantee. We integrate them in a unified execution framework that captures existing state of the art [45], enables modular design, fair comparison, and clean ablations across a well-defined design space. In summary, our contributions are as follows.

- We argue that the current bottleneck in vector joins is not just search efficiency, but a misalignment between the data's natural embedding geometry and the traditional table-centric index organization. We propose a unified offline–online co-design that resolves this misalignment; by capturing SimJoin [45] as a special case within the same pipeline, the framework also enables principled comparisons and clean ablations across the full design space, satisfying all design criteria in Table 1 for the first time.
- We introduce soft work sharing, which redefines *reusable work* as traversal effort rather than just result sets, reducing latency by up to 3x over hard work sharing while also cutting memory usage for large thresholds.
- We propose merged index, an offline encoder-centric strategy that treats the embedding space as the primary join substrate, effectively reducing initial join search to a constant-time neighborhood lookup, reducing latency by up to 44.1x.
- We identify disconnected in-range regions as a structural recall failure mode for OOD queries, and introduce an OOD-aware adaptive hybrid search that selectively bridges these regions — improving recall by up to 43% on OOD datasets while imposing no overhead on in-distribution queries.
- We validate the proposed design through comprehensive experiments. Across eight datasets, two graph indexes, and scalability tests up to 10M vectors, the results show that our three techniques are simple yet effective and complementary building blocks for efficient approximate vector join.

In the remainder of the paper, Section 2 explains related work, Section 3 defines the vector join problem and explains basic approaches, Section 4 describes our unified vector join framework and techniques, Section 5 presents experimental study, and Section 7 concludes the paper.

2 RELATED WORK

Prior work on vector search and vector join covers different parts of the overall design space, but does not provide a unified solution to approximate threshold-based vector join with broad work reuse, encoder-centric indexing, and selective handling of difficult OOD queries. Table 1 summarizes the main distinctions. Existing methods either focus on single-query ANN search (ANNS), reduce unnecessary query searches without cross-query reuse, reuse only hard in-range results under separate query/data indexes, optimize disk-based self-join and I/O reuse, or address OOD search through a specialized query-guided index rather than vector join. In contrast, our framework exposes a broader offline–online co-design space over existing graph indexes, rather than inventing yet another bespoke index structure that only works in narrow settings. To the best of our knowledge, we are the first to support fast threshold-based vector join with soft cross-query work sharing, encoder-centric merged indexing, and robust accuracy with OOD-aware adaptive traversal in one unified framework. The absence of this combination is not accidental. Query- and data-side indexes evolved from the relational table abstraction, which treats each vector set as a separate entity. This table-centric framing made it conceptually natural to build separate indexes, obscuring the fact that shared-encoder vectors already inhabit a common geometric substrate.

2.1 Non-Graph-Based ANN Index

A wide range of index structures has been developed for ad-hoc queries. Early research was dominated by Locality-Sensitive Hashing (LSH) [15, 18], which maps similar vectors into hash buckets with high probability, so queries only probe a few buckets instead of scanning all data [24]. However, this approach is not effective for high-dimensional vectors due to curse of dimensionality [7]; errors in hashing similar vectors into the same bucket increase with the dimension. Inverted file (IVF) and quantization-based approaches cluster the vector space into many buckets and search only a few relevant buckets per query [5, 19, 28]. Such ANN indexes optimized for single-query retrieval are incorporated into many vector processing systems and libraries (e.g., FAISS [16] and Milvus [42]), including the recent graph indexes below.

2.2 Graph-Based ANN Index

Graph-based ANN indexes such as HNSW [32], Vamana [27], and NSG [17] are the dominant substrate for high-performance vector search due to their strong efficiency–accuracy trade-off [16, 17, 32]. They are, however, designed primarily for single-query top-k retrieval. In-memory graph traversal is typically efficient enough that the main bottleneck becomes repeated distance computation over high-dimensional vectors, rather than disk I/O, which is exactly the regime we target in this paper. Existing graph-based ANN work also includes techniques for adaptive termination and query-specific search control. For example, DARTH [9] proposes ML-based early stopping by estimating recall during search, but it assumes a fixed-start single-query ANN regime; this is not directly portable to our work-sharing setting, where starting points vary dynamically across queries. It also targets top-k queries only, not threshold-based ones.

Recent work has also begun to address OOD queries in graph search [11, 23, 25]. RoarGraph [11] incorporates sampled historical

Table 1: Comparison of prior methods and ours along the key design dimensions of approximate threshold-based vector join.

Method	Thr.-join	Opt for reducing dist. computations	Work sharing	OOD handling	Query awareness in index construction	Index / data organization	Generalizability across indexes
Top-k ANNS	✗	✓	✗	✗	✗ (ad-hoc queries)	Data-side index only	✗
XJoin [44]	✓	✓	✗	✗	✗	Data-side index only	✓
SimJoin [45]	✓	✓	✓	✗	For query-side index only	Separate query/data index	✓
DiskJoin [12]	✓	✗	✓(I/O reuse)	✗	✗ (no separate queries)	Self-join-only index	✗
RoarGraph [11]	✗	✓	✗	✓	From sampled queries only	Query-guided data index	✗
Ours	✓	✓	✓	✓	**Jointly with data**	**Merged from construction**	✓

queries into index construction through a specialized query-guided graph, improving OOD retrieval for ANN search. Our setting is different in two ways: we study threshold-based vector join rather than single-query top-k search, and we use online hybrid search over existing graph indexes rather than building a dedicated index.

2.3 Approximate Vector Join

The most basic execution model for vector join is analogous to nested-loop or index nested-loop join, exploited by many vector systems including pgvector [1], VBASE [48], SingleStore-V [10], DuckDB [35], and more [2, 3, 42]. A naive nested-loop join computes all pairwise distances exactly, which is prohibitive at scale. Index nested-loop style execution probes a data-side ANN index independently for each query and is therefore much faster, but it repeats traversal even when many queries are similar.

XJoin [44] is an early attempt to reduce this overhead. Rather than sharing work across queries, it filters queries that are unlikely to have enough results. Thus, it still performs its ANN traversal largely independently and may degrade recall. SimJoin [45] advances this line of work. It introduces cross-query work sharing by ordering queries through a minimum spanning tree (MST) over a query-side graph index and reusing the join results of an executed query as starting seeds for its descendants. It significantly outperforms XJoin by reducing redundant traversal across similar queries rather than only pruning which queries to execute. However, its reuse is still restrictive: it caches only in-range points and discards useful traversal effort spent reaching nearby but out-range points. Moreover, because it maintains separate query- and data-side indexes, the online phase must rediscover the proximity relationships between the two sets for every join operation.

DiskJoin [12] studies a different setting of SSD-resident self-join. Its main contribution is I/O reuse through bucket scheduling and disk-aware processing, rather than in-memory graph traversal and cross-query reuse. We therefore view DiskJoin as complementary to our work rather than a direct baseline for in-memory setup.

There are also distributed and parallel solutions for similarity joins, such as MAPSS [43] and ClusterJoin [14] which are MapReduce-based join frameworks. It is an interesting future work to extend our framework to disk-based, distributed, and parallel environments.

2.4 Softening Hard Decisions in ML

Our soft work sharing was inspired by prior work in ML, which suggests that replacing a hard mechanism with a soft one can yield substantial practical gains, while still being conceptually simple. Label smoothing [38, 49] softens one-hot labels and improves generalization and calibration, knowledge distillation [8, 21] replaces hard supervision with soft teacher targets, allowing the student to exploit richer inter-class similarity information than hard labels alone, and Gumbel-Softmax [26] replaces hard categorical choices with a differentiable relaxation that substantially improves optimization for discrete decisions [31]. These examples support our intuition: just as soft labels let a learner benefit from 'near-misses' between classes, soft work sharing lets queries benefit from the traversal effort of their predecessors — effort that hard work sharing discards — broadening what information produced by search is worth reusing and generalizing naturally across indexes.

3 VECTOR JOINS

This section defines threshold-based vector join and introduces the basic execution models used throughout the paper.

3.1 Problem Definition

Definition 1. *[45] **(Vector Join)** Given two sets of vectors X and Y in $\mathbb{R}^d$, a distance function* $\mathrm{dist}(\cdot,\cdot)$*, and a distance threshold θ, the vector join is to find the set of all similar vector pairs whose distances are within the threshold, i.e., $X \bowtie_{dist<\theta} Y = \{(x,y) | x \in X, y \in Y, dist(x,y) < \theta\}$.*

We assume without loss of generality that $|X| \leq |Y|$, and refer to X as the *query* set and Y as the *data* set. We often refer to a vector as a *point* in vector space.

Unlike ad-hoc top-k search, threshold-based vector join is typically executed over a predefined batch of query vectors, similar to a join key column in relational joins. This makes it natural to assume a predefined index on queries as in [45]. In practice, vector joins are often executed in batch mode over large query sets, such as offline analytics pipelines, periodic data cleaning jobs, or hybrid vector-relational processing [10, 35, 48]. Users can join two predefined vector sets, and each vector can have attributes corresponding to the structured metadata of vectors.

Answer to: Isn't merged index used for a single join only? Wouldn't it be expensive to build a merged index for every pairwise join between $N > 2$ sets? Repeated joins over a maintained pair of sets are the norm in this workload model, not the exception. In cross-source product matching (Section 1, Appendix A), for example, a marketplace joins a maintained catalog Y against incoming seller batches X under the same encoder, exercising three reuse axes: (i) the pair is re-joined on a schedule as data evolves, with the index maintained incrementally (Section 4.3); (ii) the join runs at many thresholds on the same data—a tight θ to auto-accept high-confidence pairs, and a looser θ to generate more candidates for a reranker or human review—reusing the same *threshold-agnostic* index without rebuilding; and (iii) the same index also serves ordinary top-k search on either set (Section 5.10).

For multiple same-encoder sets, one union index with per-vector set labels serves all pairwise joins, avoiding $N(N-1)/2$ pairwise structures. The one-time offline indexing cost is thus amortized over many joins and searches, not charged to a single query.

Threshold-based joins are also more sensitive than top-k search to local density and data skew: dense regions may contain many in-range points, while sparse regions may contain very few. Fixed-k retrieval does not reflect this intrinsic structure well, since it may miss relevant matches in dense regions or return unnecessary candidates in sparse ones. This requires additional search with increased k or discarding some results [45, 48]. Threshold-based joins, in contrast, directly capture semantic proximity under a consistent decision rule.

Definition 2. ***(Approximate Vector Join)*** *Approximate vector join is to find the threshold-qualified vector pairs in an approximate way to achieve higher efficiency.*

As in ANN search, we measure the quality of an approximate vector join by *recall*, the fraction of returned join pairs relative to the exact result $X \bowtie_{\text{dist}<\theta} Y$. In the paper, we focus on graph-based ANN indexes [17, 32] since they offer the best efficiency–recall trade-off among existing vector-search families and are used by the current state of the art in approximate vector join [45].

Definition 3. ***(Graph-based Index and Search)*** *A graph-based index G is a triple (V, E, s) where V is the set of nodes representing vectors, E is the set of directed edges between nodes, forming a graph, and s is the designated entry point for search.*

We use (u, v) to denote a directed edge from node u to node v, and v is an (outgoing) *neighbor* of u. Given a query x, graph-based search starts from s and traverses the graph toward points close to x, computing query-specific distances along the way. The underlying graph (V, E) is a proximity graph.

In this paper, we use G_A to denote a pre-built graph index over vector set A. Depending on the execution mode, the framework may use separate indexes for the query and data sets, or a single merged index over both. Regardless of that choice, search is still driven by local graph traversal and repeated query-specific distance computations.

3.2 Basic Execution Models

This section summarizes the basic execution models for vector join.

3.2.1 Nested Loop Join (NLJ). The naive baseline computes all pairwise distances between X and Y, with complexity $O(|X||Y|d)$ for d-dimensional vectors. It produces the exact join result, but becomes prohibitive for large datasets and high-dimensional embeddings.

3.2.2 Index Nested Loop Join (INLJ). Each query $x \in X$ independently probes an index G_Y on the data Y. Once an in-range region is found via greedy search (navigating closer to the query), it traverses within that region via BFS [45]. This is much faster than NLJ, but it still repeats graph traversal from the fixed entry s_Y for every query, even when many queries are similar. This repeated online work is the main motivation for work sharing.

3.2.3 INLJ with Work Sharing (WS). The state of the art augments INLJ with work sharing by processing similar queries in a coordinated order and reusing information from one query for another [45]. It first constructs an MST over query graph G_X, which takes negligible time during vector join, and uses this MST to order queries starting from its root. Once a parent query completes, its approximate join results are cached and used as traversal seeds for its child queries.

The benefit of reusing the parent p's join results for its child x is approximately quantified as their distance $dist(x, p)$, from the insight that more similar queries (smaller distance) have more similar join results (larger benefits). Since the MST over the queries minimizes the sum of such inter-query distances, it maximizes the total benefits among possible query orderings. Moreover, when building this MST, not only the query-graph edges but every (s_Y, x) for each $x \in X$ is used in order to 1) ensure the connectivity of MST and 2) let a query fall back to start from the entry s_Y if it is closer to the query than the parent. s_Y is regarded as the root of MST.

Table 2: Relational analogy for vector joins.

Join type	Relational join cost		Has vector-join
	Build	Probe	counterpart?
Nested loop join	–	High	✓
Index nested loop join	High (offline)	Medium	✓
Hash join	Medium (online)	Low	✗

3.3 Why Not Hash-Join-Like Designs?

Although query ordering and work sharing reduce traversal overhead, vector join remains fundamentally different from classical relational hash join because distance computation is query-specific. Even if multiple queries reach the same point y, each query x must still compute $\text{dist}(x, y)$ and compare it with the threshold θ. This repeated high-dimensional distance evaluation is the main reason why vector joins have so far been dominated by nested-loop-like or index-nested-loop-like designs.

At the same time, this observation also suggests what is missing (Table 2). Existing methods still spend online effort to discover an initial in-range region for each query. A more aggressive design would shift part of this effort to the build phase, so that runtime search can begin from a much more relevant region, hopefully achieving *constant-time lookup, i.e., in-range-region discovery*. Our merged index moves toward this design point by encoding cross-set geometry during index construction.

4 FAST APPROXIMATE VECTOR JOINS VIA OFFLINE–ONLINE CO-DESIGN

We now present our offline–online co-design for fast approximate vector join. The key idea is to address three orthogonal issues identified in Section 1: *redundancy across similar queries*, *redundancy across joins* that repeatedly rediscovers cross-set geometry, and *reachability failure on difficult queries* whose valid matches lie in disconnected in-range regions. To do so, we combine three components in one unified execution pipeline: *soft work sharing* for online reuse, an *encoder-centric merged index* for offline work offloading, and an *OOD-aware adaptive hybrid search* for the remaining hard cases. These components are complementary and form a coherent design: the merged index is the structural foundation that eliminates redundant cross-set geometry discovery and provides the OOD signal needed for adaptive hybrid search; soft work sharing then maximizes reuse of whatever online traversal remains; and

Algorithm 1. VectorJoin

Input: Query vectors X, data vectors Y, index retriever G, threshold θ, configuration `cfg`
Output: Join result set R

Load $\{G_X, G_Y\}$ or $G_{X\cup Y}$ according to `cfg.merged`
$O \leftarrow$ OrderQueries$(G, \texttt{cfg})$
Initialize query-level cache C, result set $R \leftarrow \emptyset$
foreach $(q \in O)$ **do**
  $S \leftarrow$ SeedPhase$(q, C, G, \texttt{cfg})$
  $(I, V) \leftarrow$ GreedySearchPhase$(q, S, Y, G, \theta, \texttt{cfg})$
  $hybrid \leftarrow$ CheckHybrid$(q, G, \texttt{cfg})$
  $I \leftarrow$ ExpansionPhase$(q, hybrid, I, V, Y, G, \theta, \texttt{cfg})$
  $R \leftarrow R \cup I$; UpdateCache$(q, C, I, V, \texttt{cfg})$
return R

hybrid search fills the residual recall gap that neither offline preparation nor reuse can resolve. This stands in contrast to prior work such as [45], which optimizes the online search phase while assuming a fixed, table-centric index organization; our key departure is to change the index itself — specifically to empower the join.

4.1 Unified Execution Framework

Algorithm 1 shows the end-to-end framework. Given query vectors X, data vectors Y, graph indexes, threshold θ, and configuration parameters `cfg` (Table 3), the framework first loads either separate indexes $\{G_X, G_Y\}$ or a merged index $G_{X\cup Y}$, determines a query processing order, and initializes a query-level cache. It then processes each query in four steps: (1) construct a seed set, (2) perform greedy search to reach an initial in-range point, (3) determine whether hybrid expansion is needed, and (4) perform post-greedy expansion to collect the remaining in-range points (Figure 2 illustrates this search procedure). Finally, it updates the cache according to the selected work-sharing mode. This modular structure makes the algorithmic differences between separate vs. merged indexing, hard vs. soft work sharing, and BFS vs. hybrid expansion explicit within one common pipeline. Figure 1 maps this design space; [45] occupies the dashed lines (separate indexes, hard work sharing, BFS), while our full system extends along all three dimensions (shown in solid lines). [1]

4.1.1 Ordering and seed selection. The quality of the seed is critical. The closer the starting node is to the query, the fewer search steps are typically needed to reach an in-range region. In OrderQueries (Algorithm 2), when work sharing is enabled in hard or soft mode, queries are processed in an MST-based order so that similar queries are adjacent; otherwise they may be processed in arbitrary order. The SeedPhase (Algorithm 3) then constructs the seed set. The default seed is the global entry point of the relevant index. Depending on `cfg.work_sharing`, this seed set may be augmented with cached seeds from the parent query (hard or soft mode) or with the query point itself (self mode) for merged index. This abstraction lets separate-index and merged-index execution share the same outer pipeline while differing only in how seeds are chosen.

[1] **Answer to: Why not used the original implementation of SimJoin?** We referred to the algorithms in [45] to implement their approach within our framework but with several modifications for clarity and correctness. For example, their algorithms failed to track already-visited data points, which caused the same distance computation to be performed multiple times. As their code was not publicly available, we reimplemented their method within our framework; we achieved comparable latency (around 1ms per query on average) on the datasets we used and under small thresholds, and speedup when using their hard work sharing.

Table 3: Configuration parameters in `cfg`.

Field	Meaning
`merged`	Whether the index is merged or not.
`work_sharing`	Work-sharing mode: `none`/`hard`/`soft`/`self`.
`force_hybrid`	Always enable hybrid search.
`adapt_hybrid`	Enable hybrid search only when the query is predicted OOD.
`greedy_beam_size`	Max queue size in greedy search (default: 256).
`conv_threshold`	Threshold for determining convergence in greedy search (default: 10, sensitivity analysis in Appendix D due to space limitation).
`hybrid_beam_size`	Max queue size in hybrid search for out-range points (default: 256, sensitivity analysis in Figure 9).

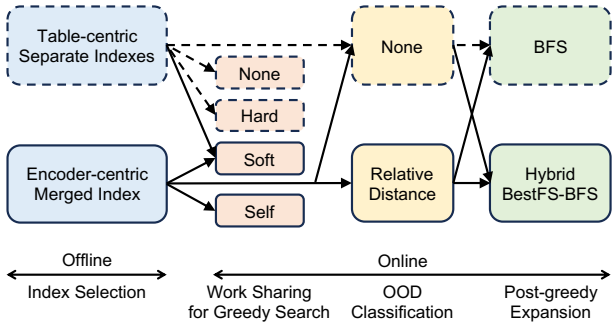


Figure 1: Design space of approx. vector join. Dashed lines are explored in existing work, whereas solid lines are our contributions. OOD classification is tied to merged index.

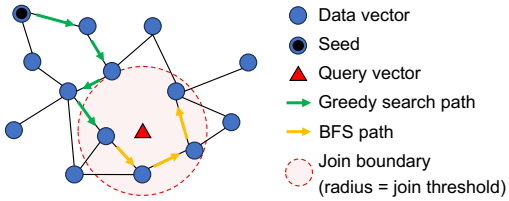


Figure 2: Search procedure for a single query: starts from a seed set, performs greedy search to reach an initial in-range point, and then performs post-greedy expansion (BFS here) to collect remaining in-range points.

Algorithm 2. OrderQueries

Input: G, `cfg`
Output: Order of queries

if (`cfg.work_sharing` $\in$ {hard, soft}) **then**
  return the MST-based order from G_X or $G_{X\cup Y}$ according to `cfg.merged`
return an arbitrary order

4.1.2 Greedy search. As shown in Figure 2, the search begins with a greedy search (Algorithm 4), implemented as a best-first traversal

Algorithm 3. SeedPhase

Input: Query q, cache C, G, cfg
Output: Seed set S

$S \leftarrow \{s_{X\cup Y}\}$ **if** cfg.merged **else** $\{s_Y\}$ /* from $G_{X\cup Y}$ or G_Y */
switch (cfg.work_sharing) **do**
  case (hard **or** soft) **do**
    $S \leftarrow S \cup C[q.parent]$ /* hard/soft work sharing */
  case (self) **do**
    $S \leftarrow S \cup \{q\}$ /* merged index */
return S

over a min-priority queue ordered by distance to the query. Starting from the seeds, the algorithm explores progressively closer nodes until one of three stopping conditions is met (Algorithm 5): (1) an in-range data point is found, (2) the best queue distance has not improved for a fixed number of iterations, or (3) in the merged-index case, the search has already probed the neighborhood of the query node that is likely to have the closest data point. Here, (2) is a simple early-stopping rule that avoids wasteful traversals when no in-range point is likely to be found, which is especially important at small thresholds. We detail (3) in Section 4.3. The greedy search returns both the in-range points found so far and the set of visited nodes to share with post-greedy expansion, so no duplicate distance computations occur.

Answer to: Why does a larger queue size not improve recall? This greedy search resembles standard top-k search [17, 32], but its goal is different: it only needs to find one in-range point rather than rank the k closest points. In top-k search, the search ends if the queue is stabilized, thus larger queues (at least size of k) can improve accuracy at the cost of more traversal. However, the queue size here matters much less, as the task is closer to top-1 search.

Algorithm 4. GreedySearchPhase

Input: Query q, seeds S, Y, G, θ, cfg
Output: Discovered in-range points I, visited points V

$I \leftarrow \{u | u \in S, dist(u,q) < \theta\}$ /* in-range data points discovered */
$V \leftarrow S$ /* visited points */
Initialize Q from S /* min-priority queue by $dist(\cdot, q)$ */
while ($Q \neq \emptyset$ **and not** StopGreedy(q, front(Q), Y, θ, cfg)) **do**
  $u \leftarrow$ popFront(Q)
  foreach (neighbor v of u with $v \notin V$) **do**
    $V \leftarrow V \cup \{v\}$; $\delta \leftarrow$ dist(v, q); insert v into Q with key δ
    if ($v \in Y$ **and** $\delta < \theta$) **then**
      $I \leftarrow I \cup \{v\}$
  trim Q to at most cfg.greedy_beam_size smallest keys
return (I, V)

4.1.3 Post-greedy expansion. The post-greedy expansion (Algorithm 7) collects remaining in-range points. By default it runs BFS over in-range nodes; when hybrid search is enabled, out-range nodes are also queued and the queue is ordered by distance rather than FIFO, so in-range regions are exhausted before out-range bridges are crossed. UpdateCache is explained in Section 4.2.

Algorithm 5. StopGreedy

Input: Query q, front element (d, u) of greedy queue Q, Y, θ, cfg
Output: Whether to stop greedy walk

if ($d < \theta$) **and** ($u \in Y$) **then return true**
if d has not been decreased for cfg.conv_threshold iterations in GreedySearchPhase **then return true**
if cfg.merged **and** $u \neq q$ **then return true**
return false

Algorithm 6. CheckHybrid

Input: q, G, cfg
Output: Whether to use hybrid search

if (cfg.merged **and** cfg.adapt_hybrid) **then**
  return (whether query q is predicted OOD)
else
  return cfg.force_hybrid

Algorithm 7. ExpansionPhase

Input: Query q, hybrid search enabled $hybrid$, initial in-range points I, visited V, Y, G, θ, cfg
Output: Final in-range points I

Initialize Q from I /* min-priority queue by $dist(\cdot, q)$ if cfg.merged, otherwise FIFO */
while ($Q \neq \emptyset$) **do**
  $u \leftarrow$ popFront(Q)
  foreach (neighbor v of u with $v \notin V$) **do**
    if (cfg.merged **and** $v \notin Y$) **then**
      continue
    $V \leftarrow V \cup \{v\}$; $\delta \leftarrow dist(v, q)$
    if ($\delta < \theta$) **then**
      insert v into Q; $I \leftarrow I \cup \{v\}$
    else
      if ($hybrid$) **then**
        insert v into Q; trim Q to at most max(cfg.hybrid_beam_size, $|Q| - 1$) keys
        /* don't discard in-range points */
return I

4.2 Soft Work Sharing (Online)

We describe our online reuse first as it is useful independently of the index organization. In Section 4.1, the online reuse corresponds to using cfg.work_sharing = hard or soft. The only algorithmic difference between the two is the cache-update policy, UpdateCache (Algorithm 8).

Algorithm 8. UpdateCache

Input: Query q, cache C, in-range points I, visited V, cfg

switch (cfg.work_sharing) **do**
  case (hard) **do**
    $C[q] \leftarrow I$
  case (soft) **do**
    $C[q] \leftarrow$ top-1 closest data point in V to q

Hard work sharing (HWS), used by prior work [45], caches all in-range points found for the current query and reuses them as

seeds for its children in the MST. This is effective when nearby queries already share many answers, but it is restrictive in two ways. First, when the threshold is small, the search may find no in-range point at all, so the cache is empty and all traversal effort is lost. Second, when the threshold is large, caching all in-range points is redundant, because the next query only needs a good seed to reach an in-range region, not the full previous answer set.

Our soft work sharing (SWS) addresses these by storing only the top-1 closest visited point, even if that point is out of range for the current query. In terms of the framework, this means setting `cfg.work_sharing = soft` and updating UpdateCache to store the best point in `visited`, rather than the set of in-range points. The key intuition is that search produces reusable information beyond final join answers: even without finding an in-range point, the closest discovered point may still be much closer to a similar future query than the global entry node of the index. Thus, SWS reuses *useful search effort* rather than only *search results*, while also keeping the cache compact. Importantly, this benefit does not depend on merged indexing; SWS is already useful under separate-index executions because it directly reduces redundancy across similar queries, mainly reducing greedy-search steps and improving efficiency without hurting recall compared to HWS.

4.3 Merged Index (Offline)

We next move beyond improving reuse under separate indexes and reduce the online effort more fundamentally by changing the index organization itself. In the framework, this corresponds to setting `cfg.merged = true`, using `cfg.work_sharing = self` by default.

The key observation is that vectors to be joined are typically produced by the same encoder [34] and already inhabit a shared embedding space. Building separate indexes forces the online phase to rediscover cross-set proximity from scratch for every join — a cost that can be offloaded to index construction. A more natural design — one that is encoder-centric — directly organizes vectors from both sets in one unified graph, encoding cross-set geometry into the index structure itself.

The question is then how to exploit this unified structure to accelerate the initial online in-range region discovery. A naive approach would be to precompute in-range regions for each query during index construction. However, this faces two fundamental obstacles: the threshold θ is not known at construction time, and even if it were fixed, a separate structure per threshold would be needed, incurring prohibitive overhead. We therefore seek a threshold-agnostic solution.

The key insight is that such a solution is already implicit in standard graph indexes, through the *relative neighborhood graph* (RNG) property [17, 39]. We *revisit* this property in the context of threshold-based vector join, showing that it is sufficient for threshold-agnostic in-range region discovery without building a dedicated index. An edge (u, v) exists in an RNG when no other point is simultaneously closer to both u and v, preserving local proximity while avoiding redundant edges. A direct consequence proved formally in Appendix C is that the top-1 closest node to any query lies in the query's local neighborhood. Therefore, once query and data vectors are organized in a single unified graph, a query node's neighborhood highly likely exposes its nearest data-side neighbor without any threshold-specific precomputation, enabling constant-time in-range region discovery: $O(md)$ for m neighbors and vector dimension d. This termination is captured by StopGreedy (Line 3). The merged index (MI) implements this idea for any off-the-shelf graph index by building $G_{X\cup Y}$ instead of G_X and G_Y (Figure 3). In this sense, the MI is analogous to a hash-join-like design point for vectors (constant-time lookup) while the build phase is offline as in index nested-loop join (Table 2). If multiple vector sets are produced by the same encoder, they can be indexed jointly in a single structure; in this paper, we focus on the common case of two sets, X and Y.



(a) Separate indexes. (b) Merged index.

Figure 3: Search procedure for two types of indexing. Same legend with Figure 2. Bottom-right query is used.

A caveat is that a query may have no neighboring data point. However, such cases are rare in practice: even on OOD datasets such as ImageNet (Section 5), where query and data distributions barely overlap, only up to 1% of queries exhibit this behavior, mostly with empty join results due to the large distance to data. Hence, we leave adding more specialized techniques (e.g., forcing cross-set neighbors) on top of MI as future work.

The offline overheads of MI are at most modest. Since graph-based indexes store a bounded number of outgoing neighbors per node, the MI preserves the same asymptotic linear space complexity as maintaining separate indexes on X and Y, and its construction time remains in the same asymptotic order. Thus, the MI is not a heavier asymptotic object; it reorganizes essentially the same indexing budget into a unified structure that enables offline work offloading. We prove this theoretically in Appendix B due to space limitation, and show in Section 5 that MI adds negligible overheads ($< 5\%$) in practice. Because the MI is built over the shared embedding space rather than for one specific threshold, it can be reused for arbitrary future threshold-based joins over the same shared-encoder vector sets. Finally, the maintenance is also simple as we can use existing index update mechanisms [32, 37].

Answer to: How is the index maintained? *Discussion on index maintenance.* Since the MI is index-agnostic, existing index maintenance mechanisms apply directly [32, 37], and maintenance optimization is largely orthogonal to this paper. A node update requires no query/data distinction—only its set label, which is consulted when the node is used in joins or searches. Incremental insertions handle streaming arrivals (e.g., seller batches in Appendix A), while periodic batch rebuilds can restore graph quality under heavy churn, exactly as for ordinary per-set indexes. The build cost itself is the standard graph-ANN construction cost one already pays for search, not a join-specific surcharge: building the MI costs essentially the same as the two per-set indexes it replaces (up to +4.5% size and +0.7% time, Section 5.5).

Discussion on filtered queries. The MI also resolves a structural limitation of work sharing over separate indexes; work sharing depends on the connectivity of the query-side graph and its induced MST. When the query set is filtered, the induced query subgraph

can be highly disconnected, and no work can be shared across disconnected components. The MI avoids this dependence on query-side connectivity altogether, because the execution is anchored in the shared query-data geometry rather than only in the query-side structure.

4.4 OOD-Aware Hybrid Search (Online)

The previous components mainly target efficiency, but an important recall failure mode remains. This is caused by a strong *locality assumption* shared by existing graph indexes and their search procedures. Once search reaches an in-range region, the remaining matches are expected to be reachable by continuing to expand locally through other in-range points. This assumption usually holds for easy in-distribution (ID) queries (e.g., SIFT in Figure 4), which is why BFS-style post-greedy expansion works well in prior work [45]. However, it can fail for difficult OOD queries, where valid matches may lie in multiple disconnected in-range regions separated by out-range points (LAION in Figure 4). In such cases, BFS becomes trapped inside the first discovered in-range region and misses others. Recovering the missing matches therefore requires selectively traversing out-range points as bridges.

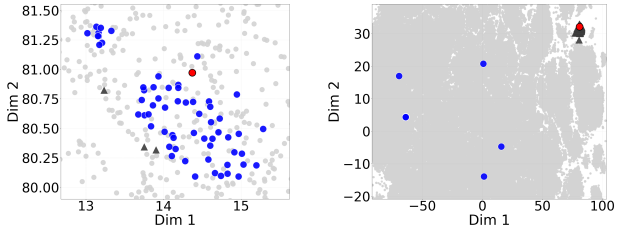


Figure 4: T-SNE visualization of SIFT (left) and LAION (right) datasets, highlighting an example query (red) and its in-range data points (blue).

Answer to: Why not use the standard beam search of top-k ANNS with a large queue? A natural alternative is best-first search (BestFS) [17], which always expands the closest point as in greedy search. However, BestFS is designed for top-k retrieval rather than threshold search. It is effective for reaching close points, but it does not exploit the fact that, once an in-range region has been reached, all connected in-range points should be enumerated. Furthermore, as mentioned in Section 3, BestFS requires k as an input which is hard to predict for a given query and threshold [29]; Figure 5 shows that k varies significantly across queries and thresholds even for the same dataset. By contrast, BFS is well aligned with threshold search, but cannot traverse out-range points. Our *hybrid search* combines the strengths of both. Like BFS, it exhaustively expands currently discovered in-range points. Like BestFS, it also uses a bounded priority queue to keep out-range points that allows best-first exploration through them when necessary (Algorithm 7), but still prioritizing in-range regions. In this way, hybrid search preserves the completeness of BFS within each in-range region while borrowing the global reachability of BestFS across disconnected regions.

The key remaining question is when to enable this extra exploration. Doing so would only add overhead for ID queries since the BFS is already sufficient. We therefore *adaptively* enable hybrid search only for OOD queries. The merged index makes this strategy

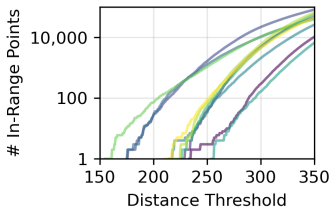


Figure 5: Cumulative distributions of number of in-range data points, one line per sampled query, on SIFT dataset.

practical by providing a lightweight OOD prediction from its query–data overlap signal in the local neighborhoods (why OOD-aware hybrid search is tied to merged index in Figure 1). We measure 1) the average distance from a query to its neighboring data points (d_1 in Figure 6) over 2) the average distance from such neighboring data points to their neighbors that are 2-hop away from the query (d_2). If $\frac{d_1}{d_2} > 1.5$, we consider that this query lies far from data and is OOD. This metric is dimensionless and encoder-agnostic, and 1.5 is a robust threshold across the datasets evaluated (Figure 6).

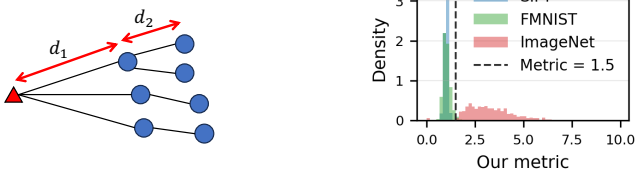


Figure 6: (Left) Features used to predict whether a query is OOD. (Right) Metric distributions for ID (SIFT, FMNIST) and OOD (ImageNet) datasets.

In the framework, this component corresponds to enabling either `cfg.force_hybrid` or `cfg.adapt_hybrid`. The latter is the default in our adaptive setting. CheckHybrid activates hybrid search only when the query is predicted OOD; otherwise, the framework uses the default BFS expansion.

Answer to: Why do MI and MI+Adapt look similar in experiments? Two behavioral consequences follow directly from this adaptivity. On in-distribution datasets, MI+Adapt falls back to BFS for nearly all queries and therefore matches MI's latency and recall; the two methods are *expected* to coincide there. On OOD datasets (e.g., ImageNet, COCO, LAION), MI+Adapt deliberately spends extra out-range traversal to bridge disconnected in-range regions, trading a bounded amount of latency for the recall that all BFS-based methods lose (Section 5).

Remark. The three components address different failure modes and therefore complement one another. Soft work sharing reduces redundant traversal across similar queries and is beneficial even under separate indexes. The merged index improves the offline organization of the search space and largely removes the online effort of greedy search. OOD-aware hybrid search then fills the remaining recall gap for difficult queries whose valid matches are disconnected. In this sense, work sharing and hybrid exploration are complementary: work sharing is most beneficial for easier, more in-distribution queries, whereas hybrid exploration is needed mainly for harder OOD queries. Furthermore, when query and data distributions heavily overlap, combining soft work sharing on top of

merged index can further improve recall, as the closest reused seed often falls within the join threshold for similar queries (Section 5.4).

5 EXPERIMENTS

This section evaluates the performance of vector join methods implemented on our framework and answers the following questions.

- Q1: Which join method shows the best performance in terms of vector join execution time and recall?
- Q2: How do index parameters affect performance?
- Q3: What is the main bottleneck?
- Q4: How large is the overhead in offline index construction?
- Q5: How well does a method scale to large data?
- Q6: How does the proximity graph affect performance?
- Q7: How does our method compare against a dedicated OOD index, and what is the trade-off for single-set searches?

5.1 Experimental Setup

5.1.1 Datasets. We evaluate our methods on eight widely used vector datasets (Table 4) that cover diverse modalities, dimensionalities, and data distributions, including vision, text, and multimodal representations. Datasets are taken from the ANN-Benchmarks [6] and VIBE [23] suites, which provide standardized query-data splits and evaluation protocols for ANN search.

Table 4: Statistics of datasets used (number of query vectors $|X|$, number of data vectors $|Y|$, dimension, mode of the degree distribution of data index G_Y, and the ratio of queries that are predicted as OOD by using our metric). The last three rows are OOD datasets.

Dataset	$\|X\|$	$\|Y\|$	Dimension	Mode	OOD-Ratio
SIFT	10,000	1,000,000	128	28	0.00%
GIST	1,000	1,000,000	960	8	1.10%
GloVe	10,000	1,183,514	200	70	0.00%
NYTimes	10,000	290,000	256	6	3.46%
FMNIST	10,000	60,000	784	13	3.03%
COCO	1,000	282,360	768	15	97.3%
ImageNet	1,000	1,281,167	640	19	97.4%
LAION	1,000	1,000,448	512	22	95.1%

SIFT and GIST consist of image descriptors commonly used for evaluating large-scale vector search systems. GloVe contains word embeddings trained on large text corpora, capturing semantic similarity in natural language. NYTimes represents document embeddings derived from news articles, reflecting real-world text retrieval workloads. FMNIST consists of Fashion-MNIST image embeddings, representing dense visual feature vectors. COCO uses multimodal image-text embeddings from the MS-COCO dataset, capturing cross-modal semantic similarity. ImageNet contains deep visual embeddings extracted from the ImageNet dataset, representing large-scale image retrieval scenarios. LAION contains CLIP-based multimodal embeddings from the LAION dataset, reflecting modern web-scale vision-language representations. Among these, COCO, ImageNet, and LAION have mostly OOD queries [23].

These datasets collectively span low to high dimensional spaces, small to million-scale data sizes, and both unimodal and multimodal embeddings, enabling a comprehensive evaluation of vector join behavior across different distributions. For each dataset, we use the standard query-data splits provided by the benchmarks by default. In-distribution queries are spread across the data manifold rather than being clustered in a single region.

Answer to: How were the distance thresholds designed? To evaluate threshold-based joins under varying selectivities, we use seven evenly spaced distance thresholds per dataset. The thresholds are chosen to cover a wide spectrum of join result sizes, from sparse joins with very few matches to dense joins where a large fraction of data vectors fall within the threshold. Table 5 reports the threshold values used for each dataset. Figure 7 shows the join sizes, illustrating the high variance in join sizes across both datasets and thresholds. Note that large thresholds with large join sizes are less likely to be used in practice, similar to using small k values (e.g., 5, 10, or 100) in typical top-k ANN searches [32, 33]. Therefore, we mainly focus on small thresholds (e.g., θ_1 to θ_3) and show the results for larger thresholds for completeness.

When the threshold is large, it is often enough to estimate the join size and approximate the results as in approximate query processing [4, 20] or LIMIT the number of outputs as in exploratory data analytics [22, 50], and full sequential scans or nested loop joins may be more efficient than index scans or index nested loop joins, as the selectivity increases in the relational context (see Appendix E for further discussion).

Table 5: Evenly-spaced distance thresholds (θ) per dataset.

Dataset	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7
SIFT	50	100	150	200	250	300	350
GIST	0.3	0.5	0.7	0.9	1.1	1.3	1.5
GloVe	0.6	0.7	0.8	0.9	1.0	1.1	1.2
NYTimes	0.1	0.3	0.5	0.7	0.9	1.1	1.3
FMNIST	500	750	1000	1250	1500	1750	2000
COCO	1.33	1.335	1.34	1.345	1.35	1.355	1.36
ImageNet	1.19	1.21	1.23	1.25	1.27	1.29	1.31
LAION	1.12	1.14	1.16	1.18	1.2	1.22	1.24

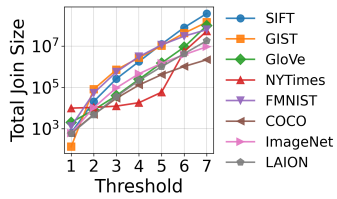


Figure 7: Join size per dataset and threshold.

5.1.2 Baselines. We implemented all baselines within our unified vector join framework to ensure a fair comparison and isolate the effect of each component. We also implemented the state-of-the-art threshold-based vector join method, SimJoin[2] [45], inside our framework and achieved similar efficiency with their paper. This allows us to fairly assess how our techniques improve over state-of-the-art work sharing.

- NLJ: Nested-loop join that computes all pairwise distances between queries and data, which is exact but impractical.
- INLJ: Index nested-loop join, each query independently performs an approximate search over the data index (`cfg.merged = false, work_sharing = none` in Section 4.1).

[2]We chose SimJoin as the main competitor that showed superior performance to XJoin (Section 2), and neither XJoin nor DiskJoin is open-sourced.

- HWS: Hard work sharing (`work_sharing = hard`) on separate indexing, corresponding to our implementation of SimJoin [45].
- SWS: Our soft work sharing (`work_sharing = soft`) on separate indexing.
- MI: Our merged index (`merged = true, work_sharing = self`).
- MI+Adapt: Merged index with our OOD-aware adaptive hybrid search (`adapt_hybrid = true`).

We use the NSG graph index [17] following [45] due to its strong efficiency-recall trade-off and fast convergence. Index parameters (e.g., graph degree and max queue size) are kept consistent across all methods to ensure comparability. We use the default parameters provided in [51] with max neighborhood size of 70 and max queue size of 256. We do not extensively tune the underlying index search algorithms beyond standard parameter settings, such as applying vector quantizations or aggressive SIMD optimizations, as our goal is to evaluate join-level optimizations rather than single-query ANN performance.

5.1.3 Metrics. We measure the latency of join execution, recall, and memory usage for the online join processing. We measure indexing time and index size for the offline phase.

5.1.4 System Configuration. For all our experiments, we use a server with Intel(R) Xeon(R) Gold 5118 CPUs and 376 GB memory. We use a single thread for evaluating vector join.

5.2 Vector Join Performance

5.2.1 Overall Results. Figure 8 shows online vector join performance. Overall, our proposed techniques – Soft Work Sharing (SWS), Merged Index (MI), and Adaptive Hybrid Search (MI+Adapt) – consistently outperform the state-of-the-art baseline Hard Work Sharing (HWS) in both efficiency and robustness of recall.

NLJ. As expected, NLJ shows consistently high latency across all thresholds and gives perfect recall.

INLJ. As expected, INLJ is significantly faster than NLJ at small thresholds, but may drop the recall noticeably for certain datasets. For NYTimes and GIST, the greedy search cannot find an in-range point due to weak data locality. The index is weakly connected (node degrees are much smaller than the other datasets, i.e., smaller Mode in Table 4) and does not effectively navigate to existing in-range points. This also occurs for OOD datasets.

HWS and SWS. While HWS reduces latency by up to 1.6x over INLJ, SWS achieves lower latency than HWS across all datasets, especially for small thresholds (θ_1-θ_3), where HWS often caches no point at all and thus fails to reuse traversal efforts. By caching the closest out-range point, SWS significantly reduces the greedy search overhead for similar queries. This improvement is most visible on SIFT, GIST, GloVe, and FMNIST, where SWS consistently reduces execution time by up to 3x compared to HWS, while maintaining similar recall. For large thresholds, SWS significantly reduces the memory usage of HWS by caching the closest data node per query, not all in-range nodes redundantly.

MI. MI significantly reduces greedy search cost by offloading the task of finding an initial in-range point to the offline index construction. As shown in Figure 8, MI achieves the lowest latency among all methods across most thresholds and datasets. This effect is particularly strong for small thresholds, where the greedy search in separate indexes often traverses many nodes before finding the first in-range point. With MI, the search frequently starts directly from a close data neighbor, reducing traversal overheads. Consequently, MI approaches constant-time behavior for finding an in-range point, yielding substantial speedups over HWS and SWS by up to 44.1x and 26.9x without sacrificing recall. Furthermore, enabling direct search from close data points often increases recall by a large margin, such as for GIST and NYTimes. Still, it often shows low recall for OOD queries; while being higher than separate-index methods, but not yet sufficient without hybrid search.

MI+Adapt. For OOD datasets (COCO, ImageNet, and LAION), MI+Adapt substantially improves recall by up to 43%. While BFS-expansion can miss disconnected in-range regions for OOD queries, MI+Adapt selectively activates the hybrid search strategy to bridge out-range walls. As a result, it consistently achieves higher recall than the others on these datasets with a trade-off of higher latency than MI. Still, the latency is lower than using separate indexes (SWS and HWS). For non-OOD datasets, MI+Adapt achieves similar latency with MI by adaptively disabling the hybrid search and falling back to BFS (Table 4). Together, these results validate the robustness claim from Section 1: the co-design maintains high recall across all eight datasets, including the structurally challenging OOD distributions.

Other methods. We omitted other methods, e.g., SWS+Hybrid (`force_hybrid = true`), MI+Hybrid, and MI+SWS whose behaviors are predictable from our analysis and showed smaller performance gaps compared to the others. For SWS+Hybrid and MI+Hybrid, forcing hybrid search for all queries increases latency for all datasets while improving recall for only OOD datasets due to the lack of adaptivity. For MI+SWS, reusing the closest visited point per query does not lead to substantial latency reduction as in INLJ vs. SWS since MI already significantly reduces greedy search steps. Still, we evaluate MI+SWS in Section 5.4 and show that it can improve recall when MI misses in-range regions.

5.2.2 Latency-recall Trade-off. Figure 9 presents the latency–recall trade-off when varying the max queue size (`greedy_beam_size` or `hybrid_beam_size` in Section 4.1) for the smallest threshold on all eight datasets. By default, the queue size controls the greedy search, while for MI+Adapt, it affects the hybrid search. As mentioned in Section 4.1, the greedy search is similar to top-1 search and thus is not affected by queue size much.

While MI and MI+Adapt together form the best Pareto curves, MI+Adapt shows a clear latency-recall trade-off by controlling the hybrid queue size. This improves recall for OOD-heavy datasets by allowing controlled traversal through out-range nodes.

5.2.3 Latency Breakdown. Figure 10 breaks down the total join latency into three parts: greedy search, BFS (or hybrid search), and the others. As expected, INLJ, HWS, and SWS are bottlenecked by the greedy search for small thresholds, finding a single in-range point, and the main goal of work sharing and merged indexing has been to reduce the traversal effort in the greedy search. MI clearly resolves this overhead. BFS dominates runtime for large thresholds due to large join sizes, seeking for all in-range points.

5.3 Query-Level Analysis

This section provides a more in-depth query-level analysis. Since per-query latency is often less than 1ms and hard to measure stably,

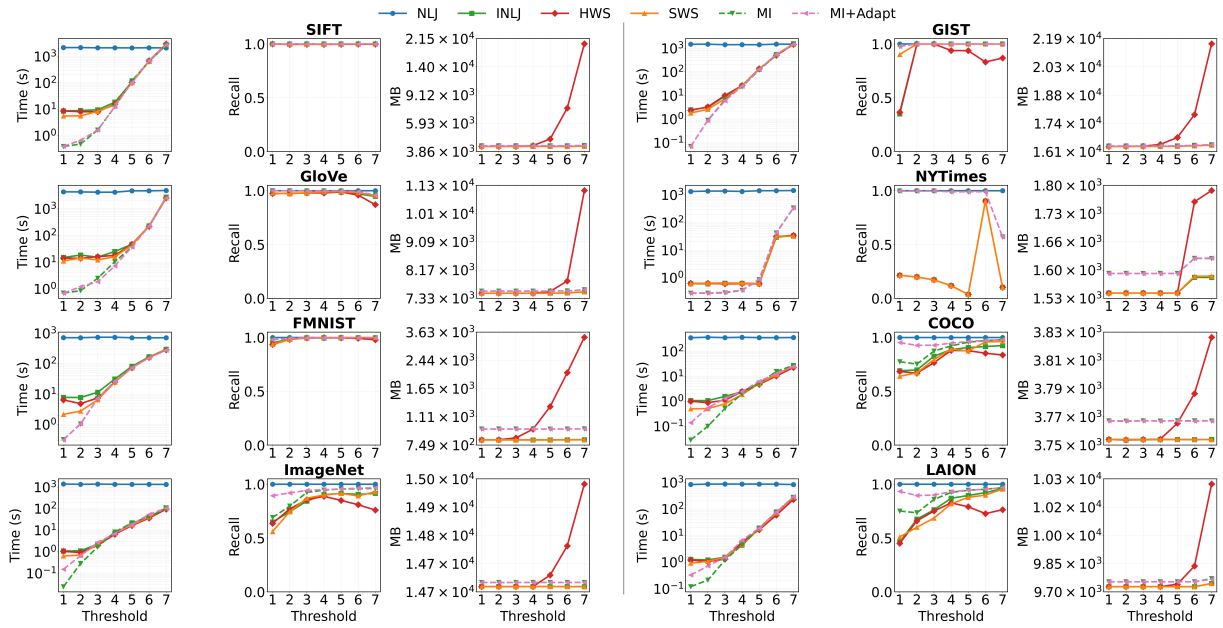


Figure 8: Vector join performance (latency, recall, and memory usage) on seven thresholds and eight datasets.

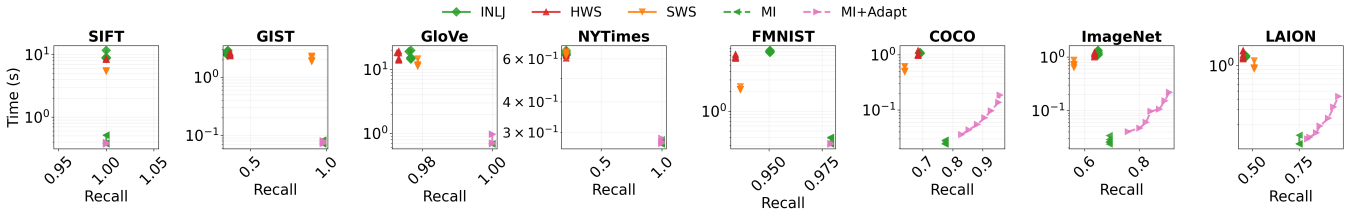


Figure 9: Latency-recall curves for varying queue sizes from 8 to 512. Smallest threshold, θ_1, is used. The queue size is defined for the greedy search phase for non-MI methods and hybrid search phase for MI+Adapt.

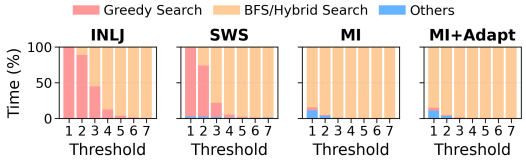


Figure 10: Latency breakdown for FMNIST into three: greedy search, BFS (or hybrid search), and the others. Similar results observed for the other datasets. HWS is similar to SWS.

we replace the latency with the number of distance computations (or simply steps) which is the main bottleneck.

5.3.1 Hard vs. Soft Work Sharing. Figure 11 groups queries by relative recall and steps for SWS vs. HWS. On SIFT, both methods achieve identical recall for all queries, while SWS uses fewer steps for most of them. On FMNIST, the same pattern largely remains: most queries still have identical recall, and SWS usually requires fewer steps, although a small number of queries favor either SWS or HWS in recall.

Figure 12 explains this behavior using the evolution of the minimum computed distance over steps on FMNIST. In both query groups, SWS generally starts closer to the query than HWS, indicating that soft sharing provides a better local initialization by reusing out-range points as well. However, for queries where SWS has lower recall, this local advantage does not always translate into better final coverage. In such cases, HWS can occasionally reach additional in-range points through larger jumps in the graph, but these jumps are artifacts of the underlying index structure rather than the objective of hard work sharing itself. This suggests that the remaining gap is not fundamentally about stronger work sharing, but about access to a better *global* index structure, which is orthogonal to work sharing.

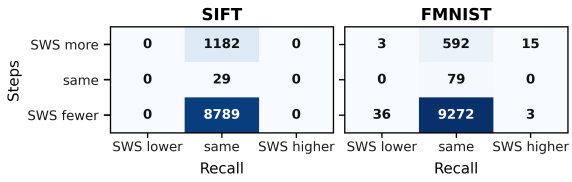


Figure 11: Query frequencies for SWS vs. HWS on SIFT and FMNIST, using the smallest threshold θ_1.

5.3.2 Separate vs. Merged Index. We next compare the separate-index design (INLJ) and the merged index (MI) at the query level. Figure 13 shows that on SIFT, both methods achieve identical recall for all queries, while MI uses fewer steps for every query. On

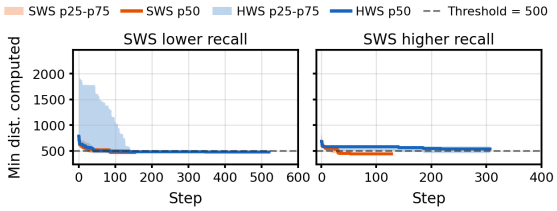


Figure 12: Min. distance computed over steps during query execution, aggregated across queries where SWS has lower/higher recall than HWS, on FMNIST in Figure 11.

FMNIST, the same pattern largely remains: MI preserves recall for most queries and still requires fewer steps in all cases. This confirms that the primary benefit of MI is to reduce online search effort by starting from a nearby data-side region already exposed by the merged graph, rather than repeatedly discovering it online.

For most FMNIST queries where MI has lower recall than INLJ, MI still finds at least one match very quickly, but misses few additional matches because the in-range regions are disconnected. Thus, MI does not fail to reach an in-range region; rather, different seeds lead MI and INLJ to different in-range components. Conversely, when MI has higher recall, it again benefits from the same local seed advantage. These results are consistent with our observations on ImageNet. Even when query and data distributions barely overlap, only 1% of queries fail to have any data point in their local merged-index neighborhoods. MI almost always reduces search steps, while the recall losses arise from disconnected in-range regions. This motivates the adaptive hybrid traversal for structurally difficult queries while trading off with more steps.

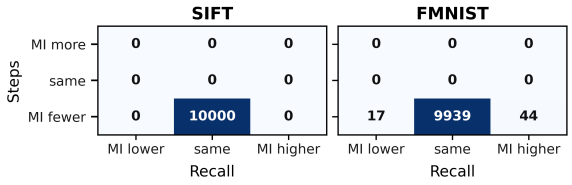


Figure 13: Query frequencies for MI vs. INLJ.

5.4 Equi-Size Join

We sample 1M query and 1M data vectors from SIFT1B[3] dataset to evaluate the methods under $|X|$ as large as $|Y|$, and four smallest thresholds from SIFT since all methods have high latencies for larger thresholds. We call this SIFT-E. In Figure 14, SWS again reduces latency by up to 2.5x compared to HWS without sacrificing recall.

Notably, MI again significantly reduces latency by 44x but shows lower recall of 0.87 than INLJ's 0.95 at the smallest threshold (for larger thresholds, both achieve high recall close to 1.0) due to the large query set that acts similarly to the out-range wall in the BFS expansion for OOD datasets. Adding SWS on top of MI (i.e., MI+SWS) improves recall to 0.9, which differs from the usual effect of SWS on separate indexes, where it mainly reduces steps. We attribute this to the strong overlap between query and data distributions in SIFT-E: the nearest data point reused across queries is often already an in-range seed for another query, so SWS helps MI recover additional relevant regions at almost no extra cost. In fact, it also yields a modest latency reduction of up to 1.2x, likely due to CPU cache effects from processing similar queries in sequence in the MST-based ordering. On the other datasets, where query-data overlap is weaker, this effect is much less pronounced, and SWS on top of MI brings smaller benefit. This suggests that MI and SWS are complementary: MI provides a better structural organization of the search space, while SWS exploits online reuse within it.

[3]http://corpus-texmex.irisa.fr

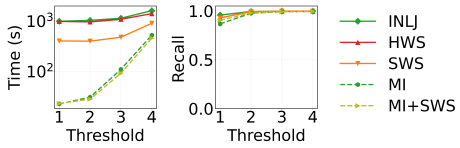


Figure 14: Vector join performance on SIFT-E using four thresholds θ_1-θ_4.

5.5 Offline Overhead

Figure 15 compares the index size and build time of using two separate indexes – one for queries G_X and one for data G_Y – against merged index $G_{X\cup Y}$. Across all datasets, merged index does not introduce much overheads compared to separate indexes, up to additional 4.5% size and 0.7% time, because the merged index uses the same graph structure and hyperparameters (e.g., max neighborhood size), and the total number of vectors is the same as $|X| + |Y|$ (refer to Appendix B for theoretical analysis). Since index construction is an offline process performed once per dataset, small additional overhead is negligible relative to the substantial runtime improvements achieved during online vector joins.

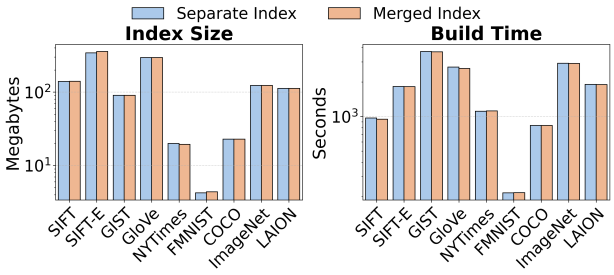


Figure 15: Offline overheads (index size and indexing time) for separate query and data indexes vs. merged index.

5.6 Scalability Test

To assess methods under varying data scales, we vary the number of data vectors $|Y|$ using the SIFT1B dataset and the smallest threshold, while keeping the number of queries at 10K. We vary $|Y|$ from 10K to 10M and sample $|Y|$ vectors from SIFT1B. 10M is the max data size we can process in memory. We leave supporting billion-scale data using multiple servers or disks as a future work. Figure 16 reports join latency as $|Y|$ increases. Recall is near 1.0 for all methods.

NLJ scales linearly with $|Y|$, as expected, since it computes all pairwise distances. INLJ scales better than NLJ (each query takes $O(\log|Y|)$ [17, 32]) but still exhibits noticeable growth due to repeated greedy searches. HWS reduces some redundancy, but its performance degrades for large $|Y|$ due to increasing cache sizes and BFS expansion costs. SWS shows improved scalability by further reducing redundant greedy search across similar queries by

caching closest out-range points. MI achieves the best scalability: as predicted by the constant-time in-range region discovery of Section 4.3, its greedy search latency scales near-horizontally, confirming the practical transition from the $O(\log|Y|)$ traversal to an $O(1)$ neighborhood probe as the data size grows.

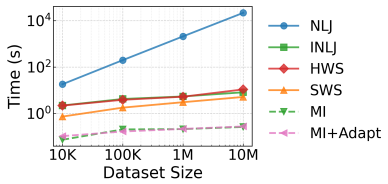


Figure 16: Scalability test with varying number of data vectors, using the smallest threshold θ_1.

5.7 Varying Index Type

So far, our experiments have used the NSG graph index [17] following [45]. In this section, we evaluate whether our techniques generalize to a different index structure, HNSW [32], which is also widely used in modern vector search systems [16, 42].

Figure 17 compares the performance of all methods using HNSW graphs instead of NSG on FMNIST and ImageNet. The ranking of methods remains consistent: MI and MI+Adapt achieve the best latency-recall trade-offs, followed by SWS, HWS, and INLJ. This shows that our framework and techniques are index-agnostic and not tailored specifically to NSG. While we set the HNSW index sizes similar to the NSG indexes we used, in general HNSW gives lower performance (especially recall) than NSG. This supports the choice of using NSG by default in [45].

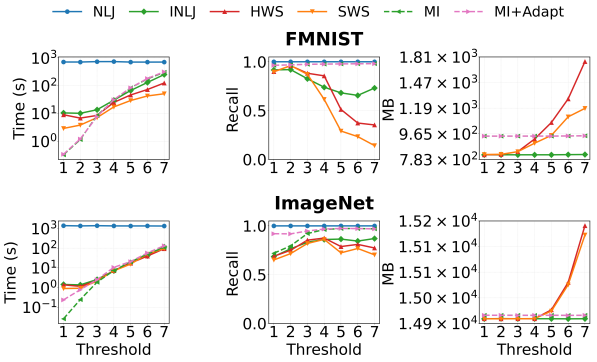


Figure 17: Vector join performance using HNSW index [32] on FMNIST and ImageNet.

5.8 Threshold Regimes and Join Size

Answer to: Why all do methods show similar performance? Small and large thresholds are two genuinely different use cases, and the dominant one is *small* thresholds with small join sizes—the analogue of small k in top-k search [17, 32]; SimJoin [45] likewise tunes its thresholds to ≤10 results per query. Figure 18(a) puts our threshold design in this perspective: our smallest thresholds per dataset correspond to SimJoin's regime, while the larger ones deliberately extend the sweep for completeness. In the small-threshold regime, work sharing and merged indexing target the greedy-search overhead, which is the main bottleneck (Figure 10), and our methods win decisively. To test this regime directly, we re-ran SIFT with ten finer thresholds calibrated to an average join size of exactly $k = 1, 2, \ldots, 10$: MI and MI+Adapt stay about an order of magnitude faster than INLJ, HWS, and SWS at recall ≈1 throughout (Figure 18(b)).

Large thresholds are a distinct, under-explored regime. There, the post-greedy expansion dominates *every* method, which must enumerate all (up to $> 10^4$) results per query (Figures 18(a) and 10), so the methods converge; this degradation is universal—[12] also reports diminishing gains at large join sizes, and very recent work targets large-k search specifically [46]—and we do not claim to solve it. We report this unfavorable regime for a fair, complete comparison rather than showing only favorable cases (Appendix E discusses a query-data switching optimization for it). Relatedly, on NYTimes at θ_7, INLJ is faster but at lower recall, and MI+Adapt's recall decreases due to (1) the low connectivity of the index over highly skewed data (low "Mode" in Table 4) and (2) the large per-query join size.

5.9 Comparison with a Dedicated OOD Index

Answer to: What if we use a SOTA OOD index for INLJ? Why do other ideas than MI give marginal performance gains? Even a dedicated state-of-the-art OOD index does not close the OOD recall gap that our adaptive hybrid search closes. On ImageNet, we strengthen INLJ in two ways: replacing its BFS expansion with our hybrid search forced on for all queries (INLJ +Hybrid), and replacing its underlying NSG index with CDMG+ [47], a state-of-the-art graph index specialized for OOD search. Both improve recall over plain INLJ but remain below MI+Adapt, while MI+Adapt is also faster at small thresholds (Figure 18(c)); we observe the same on COCO and LAION. This result also puts the gains of our online components in context: CDMG+ reports a ~3.6× speedup over its baseline [47], the same order as the ~3× latency reduction of SWS over HWS (Section 5.2).

5.10 Single-Set Search on the Merged Index

Answer to: Wouldn't the merged index degrade single-set search performance? The merged index is a *join accelerator*: it complements rather than replaces per-set indexes for search-heavy workloads. Since the merged index is larger, a single-set top-k search on it traverses out-set nodes before collecting enough in-set ones. We quantify this trade-off on SIFT (Figure 18(d)): single-set top-10 search on the merged index matches the per-set index's recall, while its relative latency depends on the ratio $r = |Y|/|X \cup Y|$: 1.3×, 2×, 5×, and 10× for $r = 0.45, 0.2, 0.06$, and 0.03. Obviously, this relative latency decreases to 1 as r increases to 1. This is a clean knob for practitioners. For join-heavy workloads, the modest single-set overhead is outweighed by the up to 44× join speedup; for search-heavy workloads with small r, one keeps a per-set index alongside, raising index maintenance cost to $(1+r)\times$ (at most 2×), optionally switching between the two adaptively. Section 6 presents our ongoing design that eliminates this trade-off altogether.

6 NN-MAP: A MATERIALIZED JOIN INDEX FOR VECTORS

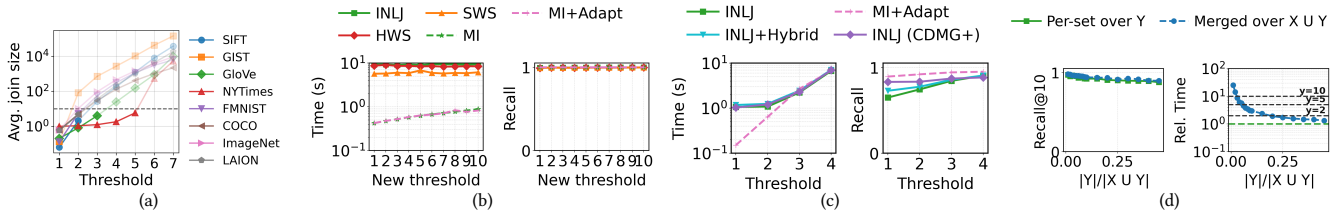


Figure 18: (a) Avg. join size k per query vs. our thresholds in Table 5; the band $k \leq 10$ is SimJoin's [45] regime. (b) Finer SIFT thresholds calibrated to $k = 1, \ldots, 10$: MI/MI+Adapt stay about an order of magnitude faster than INLJ/HWS/SWS at recall ≈1. (c) On the OOD dataset ImageNet, INLJ with forced hybrid search (no adaptivity based on the merged-index signal, Section 4.4) and INLJ over the state-of-the-art OOD graph index CDMG+ [47] both improve recall but remain below MI+Adapt. (d) Single-set top-10 search (ANNS) on a per-set index over Y vs. the merged index over $X \cup Y$ on SIFT (merged index fixed at 1M vectors): recall matches while the merged-index latency overhead decreases from ~25× to ~1.3× as $|Y|/|X \cup Y|$ grows from 1% to 45%.

Answer to: Isn't the contribution too incremental? We close our offline–online co-design by presenting *NN-Map*. NN-Map replaces the merged index, keeps the per-set separate indexes untouched and instead materializes, per join pair, the *answer* of the greedy phase itself: the top-1 nearest neighbor of each query in the other set. This retains every benefit of the merged index—threshold-agnostic reuse, offloaded in-range discovery, and the OOD signal—while eliminating its remaining trade-offs identified in Section 5.10: single-set searches run on unchanged per-set indexes at native speed, maintenance reduces to local updates, and supporting N sets costs one integer array per joined pair rather than a union/merged graph. In short, where graph indexes navigate *between vectors*, NN-Map navigates *between vector sets*.

6.1 Structure and Guarantees

Definition 4. ***(NN-Map)*** *Given two vector sets X_i and X_j from the same encoder, the NN-map $w_{i\to j}$ stores, for each $x \in X_i$, the identifier $w[x]$ of its top-1 nearest neighbor in X_j, the distance $dist[x] = \mathrm{dist}(x, w[x])$, and an OOD bit for x with respect to X_j.*

Each set keeps only its ordinary *home* (per-set) index G_{X_i}; no merged structure exists. A map is stored from the smaller set to the larger one, and the reverse join direction is served by the inverted list of the same array, so the overhead per joined pair is $\min(|X_i|, |X_j|)$ integers (plus a float and a bit). NN-Map is the vector counterpart of the classical *join index* [41]: a compact materialized access path between two relations, kept separate from either relation's own indexes. We accordingly call it a *vector join index*.

Online execution becomes a binary decision followed by the unchanged expansion phase. For a query q and threshold θ: if $dist[q] \geq \theta$, the result is certifiably empty and the query terminates without a single distance computation; otherwise, expansion starts directly from the in-range seed $w[q]$ in the home index G_{X_j}, in BFS or hybrid mode according to the OOD bit. The greedy phase does not become cheaper—it disappears.

Proposition 1. ***(Witness Property)*** *Modulo the ANN accuracy of the offline build, for every threshold θ: (i) if $dist[q] \geq \theta$, then q has no join match in X_j; (ii) otherwise, $w[q]$ is an in-range entry point for q. Moreover, the witness set $\{w[x]\}$ is minimal: removing the witness of any query loses the guarantee for some (q, θ).*

This upgrades the high-probability entry guarantee of the merged index (Appendix C) to one that holds *always* and for *all* thresholds, since nearest neighbors are threshold-independent—NN-Map remains fully threshold-agnostic. The guarantee concerns entry only: matches in disconnected in-range regions remain the job of hybrid search (Section 4.4), triggered by the precomputed OOD bit.

6.2 Build and Maintenance

Building $w_{i\to j}$ is itself an offline batch top-1 join: each $x \in X_i$ is routed through the home index of X_j with an insertion-style search that commits no edges, recording only the top-1 identifier, its distance, and the OOD score that the merged index would have derived online. The build thus costs the same asymptotic time as inserting X_i into a merged index, while the memory of the merged organization disappears entirely. Notably, our own framework bootstraps this build: the batch top-1 join is executed with SWS ordering (Section 4.2), i.e., our join algorithm constructs its own index.

Maintenance is local by construction, with no rebuild. Inserting x into the query side is one insertion search (which returns $w[x]$ as a by-product); deleting x decrements a reference count on its witness. Inserting y into the data side re-checks only the reverse-nearest-neighbor queries in y's vicinity; deleting y re-searches only the queries whose witness was y, tracked by per-node owner lists. Each operation touches work proportional to the affected witnesses, directly resolving the maintenance question raised for merged structures (Section 4.3).

6.3 Map Selection and Composition

Answer to: Isn't the solution limited to joins between only two sets? With N sets, which maps to materialize becomes an optimization on a set-level graph whose nodes are datasets and whose candidate edges are maps—the macro counterpart of index selection. Under a memory budget B, we choose a coverage $c_{ij} \in (0, 1]$ (the fraction of X_i materialized) per candidate pair (X_i, X_j), with cost $\sum c_{ij}|X_i| \leq B$ and benefit equal to the *entry-cost savings only*, since expansion is unaffected: $benefit_{ij}(c) = f_{ij}\,|X_i|\,\big[c \cdot \bar{g}_j + (1-c)(\bar{g}_j - \tilde{g}_j(c))\big]$, where f_{ij} is the join frequency, $\bar{g}_j$ the average cold-start greedy cost, and $\tilde{g}_j(c)$ the residual cost of warm-starting an uncovered query from its nearest covered query via SWS (measured by a pilot run). Under partial coverage, covered queries execute first (free entry and instant empty certificates) and then serve as anchors for the rest that use our work sharing (SWS). Since earlier anchors contribute more, the benefit is concave in c, so a marginal-benefit greedy allocation across pairs is effective.

Unmaterialized pairs are served by *composing* stored maps: for a join $X_i \bowtie X_j$ routed through X_k, the composed seed is $s = w_{k\to j}[\, w_{i\to k}[q]\,]$, and the stored distances $d_1 = dist_{i\to k}[q]$ and $d_2 = dist_{k\to j}[w_{i\to k}[q]]$ yield, with zero additional distance computations, $d_2 - d_1 \le \mathrm{dist}(q, y^*) \le d_1 + d_2$ for the true nearest $y^* \in X_j$; in particular, $d_2 - d_1 \ge \theta$ certifies an empty result. Longer h-hop compositions telescope, widening the interval by the accumulated anchor distances. Materializing a map is therefore exactly *buying this uncertainty width down to zero*, which quantifies its value inside the benefit function above. Composition is also safe: seeding from the closer of the composed seed and the default entry never does worse than the baseline greedy search. Together, selection and composition make "no $N(N-1)/2$ structures needed" a property of the design rather than a claim: one home index per set, plus a budgeted subset of integer arrays, serves all pairwise joins as well as single-set searches at no extra cost.

6.4 NN-Map as Join Statistics

The stored distances double as join statistics with an honest accuracy boundary. The count $|\{x : dist[x] < \theta\}|$ is the *exact* semi-join cardinality—the number of queries with at least one match—for any θ, which is precisely the question deduplication and contamination detection ask. Extending each entry to a top-k prefix makes the full join size exact while $dist_k(q) > \theta$, i.e., throughout the tight-threshold regime that dominates practice (Section 5.8); beyond it, a kNN-density extrapolation takes over: the head is exact, only the tail is modeled. This position differs from learned cardinality estimation for high-dimensional search, which featurizes distances to reference pivots [29]: in the known-query join setting, the references are the actual nearest neighbors and the stored distances act as answers rather than features.

These statistics feed four consumers: (i) join *direction* selection, comparing the estimated cost of either set as the outer (automating the crossover of Appendix E); (ii) join *selectivity* for vector-relational optimizers, letting vector joins participate in cost-based plans alongside relational operators; (iii) per-query execution tuning, such as hybrid activation and result-buffer preallocation; and (iv) threshold recommendation, answering "which θ yields roughly r results" by binary search on the distance CDF—the same estimate can also warn, before execution, that a chosen threshold would produce an impractically large result. In our evaluation centering on the tight-threshold regime—where entry cost is the sole bottleneck and NN-Map removes it entirely, along with N-set overhead against a union graph, maintenance throughput, and the accuracy of the composition bounds.

7 CONCLUSION

We presented an offline–online co-design for fast approximate vector joins in memory and proposed soft work sharing, encoder-centric merged index, and OOD-aware hybrid search to address three limitations of prior work: redundant traversal across similar queries, redundant discovery of cross-set geometry, and recall loss on difficult OOD queries. Our unified framework for vector join allows expressing various methods within one common pipeline, fairer comparisons, cleaner ablations, and a clearer understanding of how different design choices affect efficiency and recall. Since the framework treats the underlying graph index as a black box, it remains simple and generalizable across existing graph indexes rather than tied to a bespoke structure. Due to its simplicity, we believe this offline–online view opens up a broader design space for future vector joins in disk-based, distributed, and vector-relational settings with filters. Future work also includes extension to multi-table joins, non-graph-based indexes, and streaming data support. We further propose NN-Map that materializes the join itself rather than merging the indexes, together with its map-selection optimizer and join-statistics consumers.

REFERENCES

[1] [n.d.]. pgvector: Open-source vector similarity search for PostgreSQL. https://github.com/pgvector. Accessed: 2025-10-16.

[2] [n.d.]. Pinecone: a vector database. https://www.pinecone.io/. Accessed: 2025-10-16.

[3] [n.d.]. Qdrant: Vector Database. https://qdrant.tech/. Accessed: 2025-10-16.

[4] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. 2013. BlinkDB: queries with bounded errors and bounded response times on very large data. In *Proceedings of the 8th ACM European conference on computer systems*. 29–42.

[5] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 202–216.

[6] Erik Bernhardsson, Martin Aumüller, and Alexander Faithfull. 2025. ANN-Benchmarks: Benchmarks of Approximate Nearest Neighbor Libraries in Python. https://github.com/erikbern/ann-benchmarks/. Accessed: 2025-10-15.

[7] Christian Böhm, Stefan Berchtold, and Daniel A Keim. 2001. Searching in high-dimensional spaces: Index structures for improving the performance of multimedia databases. *ACM Computing Surveys (CSUR)* 33, 3 (2001), 322–373.

[8] Keshigeyan Chandrasegaran, Ngoc-Trung Tran, Yunqing Zhao, and Ngai-Man Cheung. 2022. Revisiting Label Smoothing and Knowledge Distillation Compatibility: What was Missing?. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.). PMLR, 2890–2916. https://proceedings.mlr.press/v162/chandrasegaran22a.html

[9] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–26.

[10] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. 2024. Singlestore-v: An integrated vector database system in singlestore. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3772–3785.

[11] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2735–2749.

[12] Yanqi Chen, Xiao Yan, Alexandra Meliou, and Eric Lo. 2025. DiskJoin: Large-scale Vector Similarity Join with SSD. *arXiv preprint arXiv:2508.18494* (2025).

[13] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered vector search: State-of-the-art and research opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5492.

[14] Akash Das Sarma, Yeye He, and Surajit Chaudhuri. 2014. Clusterjoin: A similarity joins framework using map-reduce. *Proceedings of the VLDB Endowment* 7, 12 (2014), 1059–1070.

[15] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*. 253–262.

[16] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).

[17] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2017. Fast approximate nearest neighbor search with the navigating spreading-out graph. *arXiv preprint arXiv:1707.00143* (2017).

[18] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*. 541–552.

[19] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*. PMLR, 3887–3896.

[20] Joseph M Hellerstein, Peter J Haas, and Helen J Wang. 1997. Online aggregation. In *Proceedings of the 1997 ACM SIGMOD international conference on Management of data*. 171–182.

[21] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. 2015. Distilling the Knowledge in a Neural Network. *CoRR* abs/1503.02531 (2015). arXiv:1503.02531 http://arxiv.org/abs/1503.02531

[22] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. 2015. Overview of data exploration techniques. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 277–281.

[23] Elias Jääsaari, Ville Hyvönen, Matteo Ceccarello, Teemu Roos, and Martin Aumüller. 2025. VIBE: Vector Index Benchmark for Embeddings. *arXiv preprint arXiv:2505.17810* (2025).

[24] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942* (2021).

[25] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. 2022. Ood-diskann: Efficient and scalable graph anns for out-of-distribution queries. *arXiv preprint arXiv:2211.12850* (2022).

[26] Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical Reparameterization with Gumbel-Softmax. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=rkE3y85ee

[27] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).

[28] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[29] Hai Lan, Shixun Huang, Zhifeng Bao, and Renata Borovica-Gajic. 2024. Cardinality estimation for similarity search on high-dimensional data objects: The impact of reference objects. *Proceedings of the VLDB Endowment* 18, 3 (2024), 544–556.

[30] Juan Li, Zhicheng Dou, Yutao Zhu, Xiaochen Zuo, and Ji-Rong Wen. 2020. Deep cross-platform product matching in e-commerce. *Information Retrieval Journal* 23, 2 (2020), 136–158.

[31] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. 2017. The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=S1jE5L5gl

[32] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[33] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4236–4247.

[34] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615. https://doi.org/10.1007/S00778-024-00864-X

[35] Mark Raasveldt and Hannes Mühleisen. 2019. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 international conference on management of data*. 1981–1984.

[36] Viktor Sanca and Anastasia Ailamaki. 2024. Efficient Data Access Paths for Mixed Vector-Relational Search. In *Proceedings of the 20th International Workshop on Data Management on New Hardware*. 1–9.

[37] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613* (2021).

[38] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 2016. Rethinking the Inception Architecture for Computer Vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2818–2826. https://doi.org/10.1109/CVPR.2016.308

[39] Godfried T Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern recognition* 12, 4 (1980), 261–268.

[40] Janusz Tracz, Piotr Iwo Wójcik, Kalina Jasinska-Kobus, Riccardo Belluzzo, Robert Mroczkowski, and Ireneusz Gawlik. 2020. BERT-based similarity learning for product matching. In *Proceedings of Workshop on Natural Language Processing in E-Commerce*. 66–75.

[41] Patrick Valduriez. 1987. Join Indices. *ACM Trans. Database Syst.* 12, 2 (1987), 218–246. https://doi.org/10.1145/22952.22955

[42] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*. 2614–2627.

[43] Ye Wang, Ahmed Metwally, and Srinivasan Parthasarathy. 2013. Scalable all-pairs similarity search in metric spaces. In *Proceedings of the 19th ACM SIGKDD international conference on knowledge discovery and data mining*. 829–837.

[44] Yifan Wang, Vyom Pathak, and Daisy Zhe Wang. 2024. Xling: A Learned Filter Framework for Accelerating High-Dimensional Approximate Similarity Join. *arXiv preprint arXiv:2402.13397* (2024).

[45] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Fast Approximate Similarity Join in Vector Databases. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[46] Ziqi Yin, Gao Cong, Kai Zeng, Jinwei Zhu, and Bin Cui. 2026. BBC: Improving Large-k Approximate Nearest Neighbor Search with a Bucket-based Result Collector. *CoRR* abs/2604.01960 (2026). https://doi.org/10.48550/ARXIV.2604.01960 arXiv:2604.01960

[47] Qiang Yue, Mengzhao Wang, Xiaoliang Xu, Cheng Long, Yuxiang Wang, and Jiahui Wang. 2026. Efficient and Robust Out-Of-Distribution Vector Similarity Search with Cross-Distribution Monotonic Graph. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026), 1–29.

[48] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.

[49] Zhisheng Zhong, Jiequan Cui, Shu Liu, and Jiaya Jia. 2021. Improving Calibration for Long-Tailed Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 16489–16498. https://doi.org/10.1109/CVPR46437.2021.01622

[50] Andreas Zimmerer, Damien Dam, Jan Kossmann, Juliane Waack, Ismail Oukid, and Andreas Kipf. 2025. Pruning in Snowflake: Working Smarter, Not Harder. In *Companion of the 2025 International Conference on Management of Data*. 757–770.

[51] ZJULearning. 2019. NSG: Navigating Spreading-out Graph For Approximate Nearest Neighbor Search. https://github.com/ZJULearning/nsg. https://doi.org/10.14778/3303753.3303754 Accessed: 2026-01-04.

APPENDIX

A PRACTICAL DEPLOYMENT SCENARIO

Answer to: In which applications should the proposed approach be used? A realistic end-to-end deployment scenario for our framework is cross-source product matching in e-commerce [14, 30, 40]. Suppose a marketplace maintains a large reference catalog Y, while a new seller uploads a batch of product records X. Both catalogs are encoded by the same text or multimodal encoder [45], so their vectors already lie in a shared embedding space. The system then performs a threshold-based vector join to retrieve all candidate product pairs (x, y) within distance threshold θ [44, 45], which are subsequently passed to a downstream verifier such as a rule-based matcher, a learned reranker, or human review.

This workflow highlights the practical role of our three components. First, because X and Y are produced by the same encoder, an encoder-centric merged index can be a natural indexing abstraction: it avoids artificially splitting a shared embedding space into separate query-side and data-side structures, and it offloads the online greedy-search effort into offline index construction. Second, seller uploads often exhibit locality, e.g., many products from the same brand, category, or style arrive together, so soft work sharing can reuse useful search effort across similar queries. Third, some products are structurally difficult, such as rare items, noisy titles, or records with sparse metadata; for these OOD-like queries, candidate matches may lie in multiple disconnected in-range regions, making OOD-aware hybrid search important for recall. Thus, even without introducing a new end-to-end benchmark, this scenario illustrates how merged indexing, soft work sharing, and hybrid traversal fit naturally into a practical shared-encoder vector-join pipeline. We leave evaluation on task-specific benchmarks as future work, as our focus is on the join primitive itself.

B THEORETICAL ANALYSIS FOR INDEXING OVERHEAD

We now analyze the theoretical overhead of the merged index compared to separate indexes on X and Y. The key question is whether jointly indexing both sets introduces a substantially larger indexing cost. We show that, under standard assumptions for graph-based indexes, the merged design preserves the same asymptotic space and time complexities. Therefore, the merged index should be viewed not as a heavier index structure, but as a different allocation of essentially the same indexing cost.

Theorem 1. ***(Overhead of Merged Indexing)*** *Let X and Y be the two vector sets to be joined, with $|X| = n_x$ and $|Y| = n_y$. Assume both the separate-index design and the merged-index design use the same neighbor-size bound m per vector and other construction parameters. Then the merged index preserves the same asymptotic space and time complexities as separate indexes.*

Proof. For the space complexity:

$$\underbrace{O(mn_x) + O(mn_y)}_{\text{separate indexes}} = \underbrace{O(m(n_x + n_y))}_{\text{merged index}}.$$

Hence, the merged index adds no asymptotic space overhead over separate indexes.

The construction time of the index with n vectors is

$$T(n) = O(mn \log n)\text{[32] or } O\Big(k\, n^{\frac{1+d}{d}} \log n^{1/d} + f(n)\Big)\text{ [17]},$$

which slightly differs according to the underlying index, where d is the intrinsic dimensionality (we overuse the notation d here), k is the candidate neighborhood size used in construction, and $f(n)$ is the cost of building the initial kNN graph.

Since T is convex over n, the time complexity ratio $\alpha = \frac{T(n_x+n_y)}{T(n_x)+T(n_y)}$ between the merged and separate indexes becomes the highest when $n_x = n_y = n'$. For the former $T(n)$ above of $O(mn \log n)$,

$$\alpha \le \frac{T(2n')}{2T(n')} = \frac{2mn' \log 2n'}{2mn' \log n'} = \frac{\log 2n'}{\log n'} = 1 + \frac{\log 2}{\log n'}.$$

Hence, for a sufficiently large n', the second term is negligible (e.g., 0.05 for $n' = 10^6$). For the latter $T(n)$ above, when its first term subsumes the second term $f(n)$,

$$\alpha \le \frac{T(2n')}{2T(n')} = \frac{(2n')^{1+\frac{1}{d}} \log(2n')}{2\, n'^{1+\frac{1}{d}} \log n'} = 2^{\frac{1}{d}} \cdot \frac{\log(2n')}{\log n'}.$$

Assuming $d \ge 12.9$ in typical datasets [17], $2^{1/d}$ is lower than 1.055. Including the previously computed $\frac{\log 2n'}{\log n'}$ around 1.05, $\alpha \le 1.11$. □

In the experiments, we show that the merged index adds negligible overheads in practice, less than 5% even under $n_x = n_y$.

C EXISTENCE OF CLOSEST POINT IN LOCAL NEIGHBORHOOD

Theorem 2. ***(Existence of top-1 closest node in local neighborhood under RNG)*** *Let u be a node in a relative-neighborhood graph (RNG), and let v be the closest node to u. Then $v \in N_u$, the neighborhood of u.*

Proof. By the RNG property [39], an edge (u, v) exists if and only if there is no other node w such that

$$\text{dist}(u, w) < \text{dist}(u, v) \quad \text{and} \quad \text{dist}(v, w) < \text{dist}(v, u).$$

Assume v is the closest node to u. Then there cannot exist any w with $\text{dist}(u, w) < \text{dist}(u, v)$ (Figure 19), since that would contradict the assumption that v is the closest node to u. Hence the above condition is impossible, and therefore (u, v) must be an edge in the RNG. Thus, the closest node v lies in the local neighborhood N_u of u. □

This theorem explains why probing the local neighborhood of a query node is often sufficient under the merged index. Existing graph indexes such as NSG are explicitly derived from RNG-style pruning, and HNSW- and Vamana-style graphs also approximate this locality-preserving behavior in practice [17, 27, 32]. Therefore, once a query node is inserted into the merged graph, its neighborhood is likely to expose the nearest data-side node, or at least a point very close to it. This is why the merged-index execution can often stop the greedy phase after probing the query neighborhood only.

The main caveat is that an exact RNG is too expensive to construct, and practical graph indexes only approximate this property. Accordingly, a query may occasionally have no neighboring data point in its immediate local neighborhood. However, such cases are rare in practice, and modern graph indexes such as HNSW, Vamana, and NSG still preserve this local-neighborhood intuition well

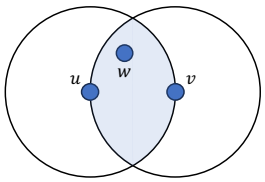


Figure 19: Illustration of a common pruning rule used in graph-based indexes. u, v, and w are vectors, and the circles are centered at u and v with radius $\mathrm{dist}(u, v)$. u and v are connected if and only if there is no w in the shaded lune such that $\mathrm{dist}(u, w) < \mathrm{dist}(u, v)$ and $\mathrm{dist}(v, w) < \mathrm{dist}(v, u)$.

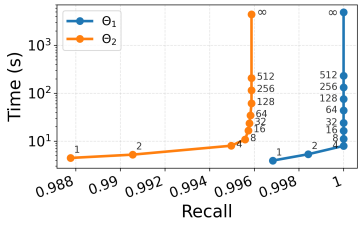


Figure 20: Sensitivity analysis on varying `conv_threshold` values (point labels) in Section 4 using SIFT dataset and two smallest thresholds.

enough to make the merged-index design effective. This also aligns with our broader design principle of keeping the solution simple and portable across existing graph indexes, rather than introducing yet another bespoke index structure specialized to one workload.

D VARYING CONVERGENCE THRESHOLD

Figure 20 shows that, the value of 4-16 for the `conv_threshold` parameter in Table 3 in Section 4 achieves a good latency-recall trade-off. This justifies our choice of using 10 as the default value.

E DISCUSSION ON LARGE THRESHOLDS AND QUERY-DATA SYMMETRY

As mentioned in Section 5, this paper focuses on small distance thresholds to align with typical top-k search with small k values. For large thresholds, all evaluated methods are bottlenecked by the post-greedy expansion (Figure 10). However, from the query-data symmetry for vector join, i.e., $X \bowtie_{dist<\theta} Y = Y \bowtie_{dist<\theta} X$, we observed that switching X and Y can reduce latency by orders of magnitude for large thresholds (and rather increase for small thresholds). This attributes to reduced random memory accesses during the post-greedy graph traversal, by sequentially iterating over the larger set Y and traversing inside the smaller graph G_X (or the X-part in $G_{X \cup Y}$). This performance gain is analogous to favoring the sequential scan over the index scan under large selectivities.